

Q1. Product of Array Except Self

```
public static int[] productExceptSelf(int[] nums) {

    int n = nums.length;
    int[] ans = new int[n];

    int prefix = 1;

    for (int i = 0; i < n; i++) {
        ans[i] = prefix;
        prefix *= nums[i];
    }

    int suffix = 1;

    for (int i = n - 1; i >= 0; i--) {
        ans[i] *= suffix;
        suffix *= nums[i];
    }

    return ans;
}

public static void main(String[] args) {

    int[] nums = {1, 2, 3, 4};

    System.out.println(
        Arrays.toString(productExceptSelf(nums))
    );
}
```

Q2. Longest Consecutive Sequence

```
public class Main {
    public int longestConsecutive(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int num : nums) {
            set.add(num);
        }
        int longest = 0;
        for (int num : set) {

            if (!set.contains(num - 1)) {

                int current = num;
                int length = 1;
                while (set.contains(current + 1)) {
                    current++;
                    length++;
                }
                longest = Math.max(longest, length);
            }
        }
        return longest;
    }
    public static void main(String[] args) {
        Main obj = new Main();
        int[] nums = {100,4,200,1,3,2};
        System.out.println(obj.longestConsecutive(nums));
    }
}
```

Q4. Daily Temperatures

```
public int[] dailyTemperatures(int[] temperatures) {  
  
    int n = temperatures.length;  
    int[] ans = new int[n];  
  
    for (int i = 0; i < n; i++) {  
  
        for (int j = i + 1; j < n; j++) {  
  
            if (temperatures[j] > temperatures[i]) {  
                ans[i] = j - i;  
                break;  
            }  
        }  
    }  
  
    return ans;  
}
```

Q5. Kth Largest Element in an Array

```
public int findKthLargest(int[] nums, int k) {  
  
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();  
  
    for (int num : nums) {  
  
        minHeap.offer(num);  
        if (minHeap.size() > k) {  
  
            minHeap.poll();  
        }  
    }  
  
    return minHeap.peek();  
}
```

Q.6 Merge K Sorted Linked Lists

```
public ListNode mergeKLists(ListNode[] lists) {

    PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);
    for (ListNode node : lists) {
        if (node != null) pq.offer(node);
    }
    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;

    while (!pq.isEmpty()) {

        ListNode node = pq.poll();

        tail.next = node;

        tail = tail.next;

        if (node.next != null) {
            pq.offer(node.next);
        }
    }
    return dummy.next;
}
```

Q.7 Course Schedule II

```
public int[] findOrder(int numCourses, int[][] prerequisites) {

    List<List<Integer>> graph = new ArrayList<>();
    int[] in = new int[numCourses];

    for (int i = 0; i < numCourses; i++) {
        graph.add(new ArrayList<>());
    }

    for (int i = 0; i < prerequisites.length; i++) {

        int a = prerequisites[i][0];
        int b = prerequisites[i][1];

        graph.get(b).add(a);
    }
}
```

```

        in[a]++;
    }

    Queue<Integer> q = new LinkedList<>();

    for (int i = 0; i < numCourses; i++) {
        if (in[i] == 0) {
            q.add(i);
        }
    }

    int[] ans = new int[numCourses];
    int count = 0;

    while (!q.isEmpty()) {

        int curr = q.poll();

        ans[count] = curr;
        count++;

        for (int next : graph.get(curr)) {

            in[next]--;

            if (in[next] == 0) {
                q.add(next);
            }
        }
    }

    if (count == numCourses) {
        return ans;
    }

    return new int[0];
}

```

Q10. Longest Increasing Subsequence

```
public int lengthOfLIS(int[] nums) {  
  
    List<Integer> tails = new ArrayList<>();  
  
    for (int num : nums) {  
  
        int left = 0;  
        int right = tails.size();  
  
        while (left < right) {  
  
            int mid = left + (right - left) / 2;  
  
            if (tails.get(mid) < num) {  
                left = mid + 1;  
            } else {  
                right = mid;  
            }  
        }  
  
        if (left == tails.size()) {  
            tails.add(num);  
        } else {  
            tails.set(left, num);  
        }  
    }  
  
    return tails.size();  
}
```

Q3 . 3Sum

```
public List<List<Integer>> threeSum(int[] nums) {

    Arrays.sort(nums);

    List<List<Integer>> ans = new ArrayList<>();

    for (int i = 0; i < nums.length - 2; i++) {

        if (i > 0 && nums[i] == nums[i - 1]) {
            continue;
        }

        int left = i + 1;
        int right = nums.length - 1;

        while (left < right) {

            int sum = nums[i] + nums[left] + nums[right];

            if (sum == 0) {

                ans.add(Arrays.asList(
                    nums[i],
                    nums[left],
                    nums[right]
                ));

                left++;
                right--;

                while (left < right && nums[left] == nums[left - 1]) {
                    left++;
                }

                while (left < right && nums[right] == nums[right + 1]) {
                    right--;
                }

            } else if (sum < 0) {

                left++;

            } else {

                right--;

            }

        }

    }

}
```

```
    }  
  }  
}  
  
return ans;  
}
```